

Modelling and optimization of distributed heterogeneous hybrid flow shop lot-streaming scheduling problem

Weishi Shao^{a,b,d}, Zhongshi Shao^{c,*}, Dechang Pi^b

^a School of Computer and Electronic Information/School of Artificial Intelligence, Nanjing Normal University, Nanjing, China

^b College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing, China

^c School of Computer Science, Shaanxi Normal University, Xi'an, China

^d Jiangsu Engineering Research Center on Information Security and Privacy Protection Technology, Nanjing, China

ARTICLE INFO

Keywords:

Distributed heterogeneous hybrid flow shop
Lot-streaming scheduling
Constructive heuristics
Iterated local search
Makespan

ABSTRACT

More enterprises are facing rapid and changing market demand and small but frequent orders, and they extend the production capability of manufacturing systems to respond to these changes. This paper studies a distributed heterogeneous hybrid flow shop lot-streaming scheduling problem (DHHFLSP) with the minimization of makespan. In the DHHFLSP, the processing capacity of each factory is different and each job can be split into several sub-lots. These sub-lots are assigned to several non-identical factories. The mixed-integer linear programming model (MILP) of DHHFLSP is established. To solve the DHHFLSP, eighteen constructive heuristics and an iterated local search algorithm (ILS) are designed. In constructive heuristics, the jobs are sorted according to several time-based heuristic rules or they are divided into several groups according to bottleneck stages, and then these jobs are assigned into factories through two NEH-based job assignment rules. In the ILS, the NEH-based heuristic (NEH_{after}) plus *Largest medium rule* is used to generate an initial solution. Two greedy insertion operators with or without critical factories are adopted to generate the perturbation solutions. Four greedy local search operators are designed to make a deep search. The influence of the parameters and main components are investigated by a comprehensive analysis. The comparisons with several related algorithms on extensive testing instances demonstrate the effectiveness and efficiency of the ILS algorithm.

1. Introduction

Nowadays, the increased global competition and dramatic changes have led many enterprises to look for better strategies to enhance competitiveness, such as increasing machines, production lines, and factories, or improving the ability of administration. However, although these strategies improve the capability of production in some ways, along with the rapid changes of consumers' requirements, the multi-kind and small-quantity production has been a wide production pattern, which has the characteristics of multi-products, small-batch, short delivery time, low inventory. Therefore, how to respond to consumers' requirements and reduce production costs is an important issue. In lot-streaming techniques, each job (or lot) is split into several sub-lots, and overlapping operations in different manufacturing stages are executed to accelerate production and increase machine utilization (Chang & Chiu, 2005). For example, in semi-conductor production, a job usually consists of many identical electronic elements. It is not necessary

to wait for all items in the job to be completed before moving to the down-stream machines, so a job can overlap its operations between successive machines by dividing it into a lot of sub-lots and moving the completed portion of the sub-lots to down-stream machines (Pan & Ruiz, 2012). Therefore, how to get an efficient lot-streaming schedule is also important. This paper focuses on obtaining the best sub-lot allocation, i. e., sub-lot starting and completion times, and the best job sequence (Pan, Fatih Tasgetiren, Suganthan, & Chua, 2011).

In recent years, more and more group companies establish many factories in the world to increase the production capability, respond to markets, and reduce risk. Generally, these factories have different features, such as machines, production efficiency, worker, cost, and so on. It leads to a distributed production scheduling problem in heterogeneous factories. For example, BYD is a new enterprise with three major industrial groups of IT, automobile, and new energy. It establishes many production bases in Guangdong, Beijing, Shaanxi, Shanghai, Tianjin province of China. These bases have different

* Corresponding author.

E-mail address: shaozhongshi@hotmail.com (Z. Shao).

<https://doi.org/10.1016/j.eswa.2022.119151>

Received 2 May 2022; Received in revised form 15 October 2022; Accepted 24 October 2022

Available online 1 November 2022

0957-4174/© 2022 Elsevier Ltd. All rights reserved.

production lines, workers, environments, products, and so on. A high-efficient scheduling method is required to arrange production tasks. Motivated by the development of distributed production, this paper considers a distributed heterogeneous hybrid flow shop lot-streaming scheduling (DHHFLSP). As seen in Fig. 1, the production system composes of several non-identical factories and each factory includes a hybrid flow shop where each stage includes several parallel machines. It permits the jobs can be split into several sub-lots. To the best of our knowledge, although the lot-streaming scheduling problem in flow shop or job shop problems are extensively studied (Božek & Werner, 2018; Buscher & Shen, 2009; Han, Gong, Jin, & Pan, 2019; Zhang, Yin, Liu, & Linn, 2005), the DHHFLSP is not studied before our work. More details about distributed production scheduling can be seen literature (Fu et al., 2021; Lohmer & Lasch, 2021).

Compared to previous research work, the contribution of this paper can be summarized as follows. (1) The DHHFLSP with the makespan criterion is presented and modeled by using the MILP model that is not studied before our study. In the distributed heterogeneous hybrid flow shop, the different quantities of parallel machines and the different processing speeds of machines are considered. The lot-streaming scheduling problem is integrated into the distributed heterogeneous hybrid flow shop. (2) We extend the shortest processing time, Johnson's rule, large medium rule, largest small rule, longest processing times till bottleneck stage, KK rule by using the properties of DHHFLSP, i.e. the setup time and processing time of sub-lots. These rules provide a promising priority sequence for constructive heuristics. Two NEH-based job assignment rules are proposed to establish a promising heuristic solution. In these heuristics, after a job is assigned into a position, the jobs before (or after) this position are re-assigned to the positions after (or before) this position. These two heuristics reduce the search spacing compared to the previous NEH-based algorithms. (3) An iterated local search algorithm (ILS) for solving the DHHFLSP is presented based on the properties of DHHFLSP. The large medium rule plus NEH_{after} is used to generate a promising solution. In the perturbation part, we employ two greedy insertion operators based on the critical factory which can ensure the perturbation solutions around the best solution. In the local search part, four local search methods including the swap and insertion operators are employed based on the critical factory. These local search methods consider a critical property in the DHHFLSP, i.e. removing a job from the critical factory may not decrease the makespan, since the job may not be in the critical route. The experimental results demonstrate the effectiveness and efficiency of the proposed constructive heuristics and ILS.

The following sections are organized as follows. Section 2 reviews the recent works about lot-streaming scheduling in hybrid flow shop and distributed hybrid flow shop scheduling. Section 3 defines the DHHFLSP. Section 4 presents several constructive heuristics including priority sequence generation rules and job assignment rules. Section 5 proposes an iterated local search algorithm. Section 6 lists the experimental results including the parameters calibration, the analysis of components, the comparison of constructive heuristics and ILS with other algorithms. Section 7 summarizes this paper and provides some future work.

2. Literature review

2.1. Lot-streaming scheduling in hybrid flow shop (HLFSP)

Lot-streaming scheduling has been paid much attention in recent years, especially in flow shop (Han et al., 2019; Pan & Ruiz, 2012; Pan et al., 2011) and job shop (Božek & Werner, 2018; Buscher & Shen, 2009). However, the HFSP is also a realistic scheduling problem, but the literature about the HFSP with lot-streaming problems is not rich. Zhang et al. (2005) considered the lot-streaming scheduling in two-stage HFSP with $m-1$ identical machines at the first stage and a single machine at the second stage. Cheng and Sarin (2013) considered a two-stage HFSP with lot-streaming where the first stage includes one machine and the second stage includes two parallel machines. Naderi and Yazdani (2014) considered a HFSP with sub-lots and sequence dependent setup times and presented an imperialist competitive algorithm that employs imperialist behavior, imperialist competition and colony improving mechanisms. Zhang et al. (2017) considered the objective of the total flow time and proposed a modified migrating birds optimization algorithm where they adopted a shortest waiting time rule that can schedule the jobs concurrently arriving at stages more reasonably. Chen, Cheng, and Chou (2020) studied an energy-efficient hybrid flow shop lot-streaming scheduling (HLFSP) with the objective of the makespan and electric power consumption and employed NSGAII to solve this problem. Qin, Zhuang, Liu, and Tang (2019) considered the lot sizing, calendar constraints and sequence-dependent setup times in hybrid flow shop scheduling and presented a two-stage ant colony algorithm to solve PCB assembly shop scheduling problem. Nejati, Mahdavi, Hassanzadeh, and Mahdavi-Amiri (2016) studied an assembly scheduling where the first stage includes m machines and the second stage includes several assembly machines under lot sizing environment. A mixed algorithm combining a genetic algorithm and a simulated annealing algorithm was

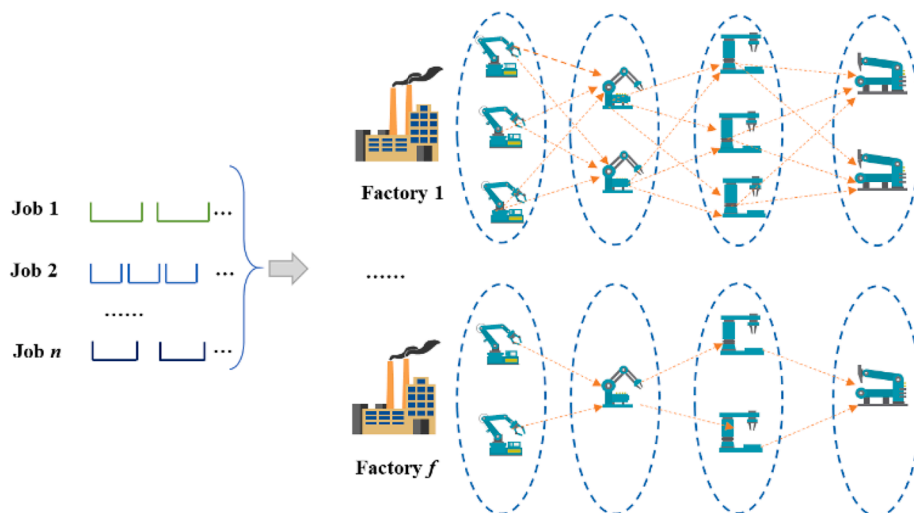


Fig. 1. An example of distributed heterogeneous hybrid lot-streaming flow shop scheduling.

presented to solve this problem. Zohali, Naderi, Mohammadi, and Roshanaei (2019) considered an integrated economic lot-sizing and sequencing problem in the HFSP with unlimited intermediate buffers where two decisions were considered, i.e. the manufacturing sequences of products and their production quantity. Li et al. (2020) studied a multi-objective HFSLP with variable sub-lots and considered three objectives, i.e. the penalty caused by the average sojourn time, energy consumption and earliness and tardiness. A MOEA/D with problem-specific heuristics was presented.

2.2. Distributed hybrid flow shop scheduling (DHFSP)

Recently, the research on the DHFSP has been gained more attention. Most of the researchers focus on problem models, properties, objectives, and meta-heuristics. The following paragraphs list the recent works about the DHFSP.

Ying and Lin (2018) studied a DHFSP with several processors where several processors co-process a job at the same time at each stage, and designed a self-tuning iterated greedy algorithm whose key is an adaptive destruction and construction procedure. Hao et al. (2019) proposed a hybrid brain storm optimization algorithm to solve DHFSP with the makespan objective. For the same problem, Cai, Zhou, and Lei (2020) designed a dynamic shuffled frog leaping algorithm that employs dynamic global search, multiple neighborhood search, and destruction-construction. Lei and Wang (2019) proposed a shuffled frog-leaping algorithm for solving the DHFSP. Wang and Wang (2019) studied the DHFSP with the total tardiness objective and presented an iterated greedy algorithm that employs a variable neighborhood search-based local intensification that uses several problem-specific neighborhood structures. Shao, Shao, and Pi (2020) presented a multi-neighborhood iterated greedy algorithm that uses a multi-search construction to increase the diversity and a multi-neighborhood to enhance local searching ability. They considered several identical factories including the same number and the capability of machines. Li, Li, Gao, and Meng (2020) studied a distributed heterogeneous hybrid flow shop scheduling problem (DHHFSP) in which the processing capability of each machine is non-identical. An improved artificial bee colony (ABC) algorithm was presented which employs some local exploitation techniques around critical factories to generate solutions, but the efficiency of these techniques is not high and the searching is not deep for a solution. After then, Li et al. also presented an ABC for the DHHFSP with the identical factories in literature (Li, Li, Gao, et al., 2021), but the techniques and the mathematical model are similar to (Li et al., 2020). Wang and Wang (2020) also presented a bi-population cooperative memetic algorithm, which integrates collaborative initialization, bi-population cooperation and local intensification. Zheng, Wang, and Wang (2020) studied a multi-objective fuzzy DHFSP with fuzzy processing times and fuzzy due dates and presented a cooperative coevolution algorithm that integrates the estimation of distribution algorithm and iterated greedy search. Shao, Shao, and Pi (2021c) studied the multi-objective DHFSP with the objective of minimization of makespan, total weighted earliness and tardiness, and total workload. They presented a multi-objective evolutionary algorithm where some multiple neighborhood local search operators based on the objectives were designed to generate solutions. Shao, Shao, and Pi (2021a) studied the DHFSP with non-identical time of use electricity tariffs and presented an ant colony optimization which employs the nodes in the route to denote the jobs and imitate the ant behaviors to generate solutions. Shao, Shao, and Pi (2021b) considered the no-wait constraint in DHFSP and presented many constructive heuristics by considering several time-based indicators. Lu, Gao, Yi, and Li (2020) studied a motorcycles production line that includes several permutation flow shop and hybrid flow shop lines. Wang and Wang (2021) studied the energy-aware DHFSP with the makespan and the total energy consumption. They proposed a cooperative memetic algorithm (CMA) with a reinforcement learning based policy agent. The

reinforcement learning is used to select improvement operators. Enda, Ling, and Jingjing (2021) considered the energy-aware DHFSP with multiprocessor tasks and proposed a multi-objective evolutionary algorithm based on decomposition where a cooperative search is designed to produce new solutions. Li, Chen, Duan, and Mou (2022) studied the DHFSP with variable speed constraints in a prefabricated system, and proposed a knowledge-based multi-objective evolutionary algorithm that integrates crossover operator, SAR-based local search heuristic, and initialization approach. Li, Li, Zhang, et al. (2021) studied the DHFSP with type-2 fuzzy processing time and setup time with the objective of minimizing minimize the maximum type-2 fuzzy completion time among all factories.

Some researchers have noticed lot-streaming scheduling in distributed production scheduling. Most of the researches focus on the distributed flow shop lot-streaming scheduling problem (DFLSP). Duan, Meng, Chen, and Pan (2018) presented a discrete artificial bee colony for the DFLSP which uses a multi-list-based swap and insertion operators to generate solutions. Meng and Pan (2021) considered the carryover sequence-dependent setup time in the DFLSP. Several constructive heuristics and an artificial bee colony algorithm were proposed for solving this problem. Pan, Gao, Li, and Wu (2022a) modeled the mathematical model of DFLSP and designed several problem-specific features and knowledge-based local strategies, and then they employed five meta-heuristics including particle swarm optimization, genetic algorithm, harmony search, artificial bee colony, and Jaya algorithm to solve the DFLSP. Then, Pan, Gao, Li, and Wu (2022b) also studied the multi-objective version of DFLSP with the makespan and the total energy consumption.

From the above literature, it can be seen that the research about the DHFSP with complex constraints or real production complex is limited. Most methods pay attention to meta-heuristics, but the constructive heuristic is also limited.

2.3. Gap and discussion

Based on the above review, we find some interesting limitations.

- (1) To the best of our knowledge, the distributed hybrid flow shop lot-streaming scheduling has not been studied. In most studies on the distributed hybrid flow shop scheduling, the jobs cannot be divided into sub-lots. However, the multi-item and small-lot settings have widely in modern production, and this production system needs to be changed to fit the new settings.
- (2) The constructive heuristics for lot-streaming scheduling are not gained efficient attention, especially priority sequence generation rules. However, in real production scheduling, constructive heuristics can give a promising schedule within a short running time.

3. Problem statement

The MILP model of DHHFSP is shown as follows.

Indexes:

- j Index of job $j \in J$.
- f Index of factory $f \in F$
- i Index of machine, $i \in M_{f,k}$
- l Index of sub-lot of job j , $l \in L_j$
- k Index of stage, $k \in S$

Parameters:

- N Number of jobs.
- S Number of stages.
- F Number of factories.

$m_{f,k}$ Number of machines at stage k in factory f .
 L_j Number of sub-lots of job j .
 J Set of jobs, $J = \{1, 2, \dots, N\}$
 S Set of stages, $S = \{1, 2, \dots, S\}$
 F Set of factories, $F = \{1, 2, \dots, F\}$
 $M_{f,k}$ Set of machines at stage k in factory f .
 L_j Set of sub-lots of job j , $L_j = \{1, 2, \dots, L_j\}$
 $M_{f,k,i}$ Machine i at stage k in factory f .
 $t_{j,k}$ Processing time of sub-lot of job j at stage k .
 $setup_{k,j'j}$ Setup time from job j' to job j at stage k .
 $setup_{k,jj}$ Setup time between sub-lots of job j at stage k .
 $setup_{fj,k}$ Setup time of job j at the first position of stage k .
 $sp_{M_{f,k,i}}$ Speed coefficient of machine $M_{f,k,i}$.
 U Enough large number.

Variable:

$x_{f,j}$ If job j is processed in factory f , $x_{f,j} = 1$; otherwise, $x_{f,j} = 0$
 $y_{f,k,j,i,q}$ If job j is processed at position q of machine i stage k in factory f , $y_{f,k,j,i,q} = 1$; otherwise, $y_{f,k,j,i,q} = 0$
 $s_{j,l,k}$ Start time of sub-lot l of job j at stage k .
 $c_{j,l,k}$ Completion time of sub-lot l of job j at stage k .
 $sq_{f,k,i,q}$ Start time of position q of machine i at stage k in factory f .
 $cq_{f,k,i,q}$ Completion time of position q of machine i at stage k in factory f .
 C_{max} Maximum completion time of all factories.

3.1. Problem definition

The DHHFLSP can be described as follows. The manufacturing system consists of F heterogeneous factories and each factory includes a hybrid flow shop that contains a series of S processing stages. Each stage k has several non-identical parallel machines, and there is at least one stage that includes more than one machine. The speeds of machines are different. A speed coefficient sp is employed to reflect the speed of a machine. A set of jobs are assigned to F factories to be processed following the same route from the first stage to the last stage S . The equal and consistent lot sizes are used, i.e. each job j can be split into L_j sub-lots with the same size and L_j is the same for all machines. The sub-lot size L_j of each job has been given. All sub-lots belonging to the same job should be consecutively processed so that no intermingling with sub-lots from a different job is permitted. It should be noted that this paper focuses on the lot-streaming scheduling but not lot sizing problem. Once the processing of a sub-lot is completed, it can be transferred to the next machine immediately. A sub-lot of the job should be processed on a stage during a period of basic time $t_{j,k}$ without break. Then, due to the different speeds of machines, the processing time of a sub-lot on a machine M at a stage can be computed by $t_{j,k}/sp_M$. A inseparable sequence-independent setup time at stage k is needed for each sub-lot: (1) the first sub-lot of job j processed in the first position of machine i requires a setup time $setup_{fj,k}$; (2) the first sub-lot of job j following the sub-lot belong to another job j' on machine i requires a setup time $setup_{k,j'j}$; (3) the other sub-lots of job j requires a setup time $setup_{k,jj}$ before handled on machine i . Once a sub-lot is assigned to a certain factory, all its procedures should be completed in the current factory. At any time, each machine can process at most one sub-lot and each sub-lot can be processed on at most one machine. Each sub-lot is independent and available at time zero. The transportation time between consecutive machines is neglected. The buffer capability is infinite. The idle time is allowed for machines and sub-lots can wait between successive stages. The task of this scheduling problem is to optimize the assignment of sub-lots to factories, selection of machines, and

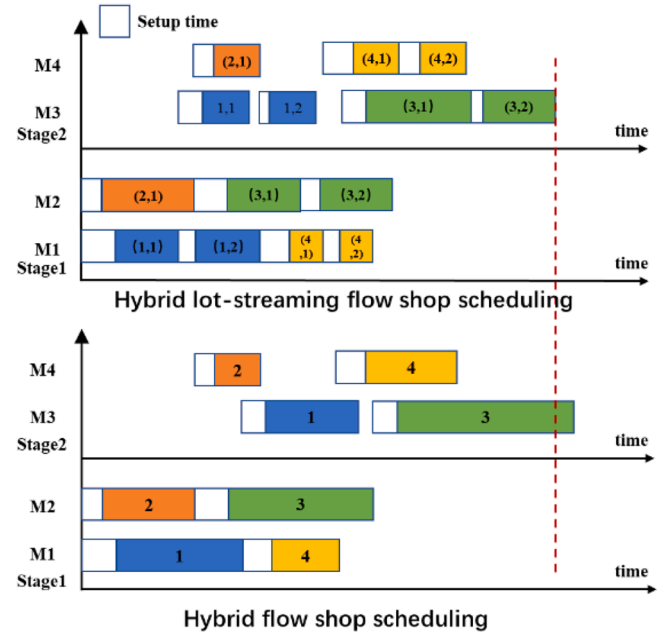


Fig. 2. An example for the difference between hybrid flow shop scheduling with and without lot-streaming.

determination of processing order of sub-lot at each stage so that the makespan is minimized. Fig. 2 shows an example of hybrid flow shop scheduling with or without lot-streaming in a factory. It can be observed that overlapping operations reduce the total completion time.

The production decisions often include the aggregate planning level and the detailed scheduling level (Drexel & Haase, 1995). They are two separate different problems that exist in different departments (planning department and scheduling department) of production enterprises in real life. Lot-sizing solves the determination of size of production lots for one or more items, but lot-streaming scheduling determines the sequence of these sub-lots. This paper focuses the second one and the numbers of sub-lots of each job have been given. That is to say, the primary purpose is to simultaneously obtain the best sub-lots schedule, i.e., sub-lot starting and completion times. Of course, there are some literatures that have paid attention to integrating of lot-sizing and scheduling (Lee & Lee, 2020; Martínez, Adulyasak, Jans, Morabito, & Toso, 2019).

3.2. MILP model of DHHFLSP

Based on literature (Y. Li et al., 2020), the MILP model of DHHFLSP is defined. Li et al. established the MILP models of DHFSP that defines the constraints of jobs' position and the precedence relationship between jobs. We refer this model to define the precedence relationship between jobs, i.e. Eqs. (2)–(8). Based on this model, we add Eqs. (9)–(16) to define the starting time and the ending time of the sub-lots. The MILP model of DHHFLSP is shown as follows.

Objective

$$f = C_{max} \quad (1)$$

s. t.

$$\sum_{f \in F} x_{f,j} = 1, j \in J \quad (2)$$

$$\sum_{i \in M_{f,k}} \sum_{q \in J} y_{f,k,j,i,q} - x_{f,j} = 0, j \in J, k \in S, f \in F \quad (3)$$

$$\sum_{j \in J} y_{f,k,j,i,q} \leq 1, \quad k \in S, \quad f \in F, \quad i \in M_{f,k}, \quad q \in J \quad (4)$$

$$\sum_{j \in J} y_{f,k,j,i,q} \geq \sum_{j' \in J} y_{f,k,j',i,q+1}, \quad k \in S, \quad f \in F, \quad i \in M_{f,k}, \quad q \in J / \{N-1\} \quad (5)$$

$$sq_{f,k,i,q+1} + U \times (1 - y_{f,k,j,i,q+1}) \geq cq_{f,k,i,q} + \sum_{j' \in J} setup_{k,j',j} \times y_{f,k,j',i,q} \quad (6)$$

$$j \in J, \quad k \in S, \quad f \in F, \quad i \in M_{f,k}, \quad q \in J / \{N\}$$

$$sq_{f,k,i,q+1} \geq cq_{f,k,i,q}, \quad j \in J, \quad k \in S, \quad f \in F, \quad i \in M_{f,k}, \quad q \in J / \{N\} \quad (7)$$

$$sq_{f,k,i,1} \geq setup_{f,j,k} - U \times (1 - y_{f,k,j,i,1}), \quad j \in J, \quad k \in S, \quad f \in F, \quad i \in M_{f,k} \quad (8)$$

$$sq_{f,k,i,q} \leq s_{j,l,k} + U \times (1 - y_{f,k,j,i,q}), \quad j \in J, \quad k \in S, \quad f \in F, \quad i \in M_{f,k}, \quad q \in J \quad (9)$$

$$sq_{f,k,i,q} \geq s_{j,l,k} - U \times (1 - y_{f,k,j,i,q}), \quad j \in J, \quad k \in S, \quad f \in F, \quad i \in M_{f,k}, \quad q \in J \quad (10)$$

$$cq_{f,k,i,q} \leq c_{j,l,j,k} + U \times (1 - y_{f,k,j,i,q}), \quad j \in J, \quad k \in S, \quad f \in F, \quad i \in M_{f,k}, \quad q \in J \quad (11)$$

$$cq_{f,k,i,q} \geq c_{j,l,j,k} - U \times (1 - y_{f,k,j,i,q}), \quad j \in J, \quad k \in S, \quad f \in F, \quad i \in M_{f,k}, \quad q \in J \quad (12)$$

$$c_{j,l,k} = s_{j,l,k} + \sum_{f \in F} \sum_{i \in M_{f,k}} \sum_{q \in J} t_{j,k} / sp_{M_{f,k},j} \quad j \in J, \quad k \in S, \quad l \in L_j \quad (13)$$

$$s_{j,l,k+1} \geq c_{j,l,k} + setup_{k,j,j}, \quad j \in J, \quad k \in S / \{S\}, \quad l \in L_j \quad (14)$$

$$s_{j,l+1,k} \geq c_{j,l,k} + setup_{k,j,j}, \quad j \in J, \quad k \in S, \quad l \in L_j / \{L_j\} \quad (15)$$

$$C_{\max} \geq c_{j,l,j,S}, \quad j \in J \quad (16)$$

$$sq_{f,k,i,q}, \quad cq_{f,k,i,q}, \quad s_{j,l,k}, \quad c_{j,l,k} \geq 0, \quad j \in J, \quad k \in S, \quad f \in F, \quad i \in M_{f,k}, \quad q \in J, \quad l \in L_j \quad (17)$$

$$x_{f,j}, \quad y_{f,k,j,i,q} \in \{0, 1\}, \quad j \in J, \quad k \in S, \quad f \in F, \quad i \in M_{f,k}, \quad q \in J \quad (18)$$

Eq. (1) defines the objective, i.e. makespan. Constraint set (2) ensures that the job must be assigned to only one factory. Constraint set (3) ensures that each job can only be processed on one machine at each stage of one factory. Constraint set (4) ensures that the job can only occupy-one position on the machine. Constraint set (5) guarantees that each job selects a machine based on the successive position of the machine and ensures the previous position of the machine is selected. Constraint set (6) denotes the requirement of setup time between two consecutive jobs. Constraint set (7) ensures that the starting time of the successive position of the machine should be larger than the completion time of the previous position of the machine. Constraint set (8) expresses the job processed in the first position of machine requires a setup time. Constraint sets (9) and (10) define the starting time of each position of the machine, i.e. the starting time of the first sub-lot processed in the machine. Constraint set (11) and (12) define the completion time of each position of the machine, i.e. the completion time of the last sub-lot processed in the machine. Constraint set (13) defines the completion time of the sub-lot as the processing time plus the starting time. Constraint set (14) ensures the starting time of the sub-lot at the next stage should be larger than the completion time of the sub-lot at the previous stage plus a setup time. Constraint set (15) ensures the starting time of the successive sub-lot should be larger than the completion time of the previous sub-lot plus a setup time. Constraints set (16) defines the completion time of all factories. Constraint set (17) ensures that the starting time of each position and sub-lot is not less than 0. Constraint set (18) defines the decision variables.

4. Constructive heuristics

Constructive heuristics can generate a promising solution with less computing time. The NEH heuristic is the best constructive heuristic for solving flow shop scheduling. For distributed flow shop scheduling, the applications of NEH come from the works of [Naderi and Ruiz \(2010\)](#). They proposed two simple rules. One is assigning the jobs to the factory with the minimum current makespan, not including this job. The other is allocating the jobs to the factory that has the minimum makespan after including this job. The core idea of NEH has been widely used to solve the distributed shop scheduling problem. [Ruiz, Pan, and Naderi \(2019\)](#) presented an enhanced NEH where after inserting a job into the best position among all the factories, either the previous or the following job (at random) is extracted and tested in all positions in the same factory. [Shao, Pi, and Shao \(2017\)](#) re-performed a greedy insertion on all jobs in the same factory. The main characteristics of these NEH heuristics can be summarized as follows. Most of NEH heuristics consist of a priority sequence of jobs, i.e. the determination order of jobs, and a greedy insertion that assigns the jobs to the factories. To increase the depth of search, if a job has been assigned to a certain factory, then a simple local search is applied to the current factory. This procedure can provide a promising solution compared to simple rules. More insertions mean much computing time, so reducing the searching time and increasing the effective search is the key. For the DHHFLSP, this paper designs six priority generating rules based on the setup time and processing time and two NEH-based greedy insertions to construct several heuristics. The sorting heuristic rules are shown as follows.

- (1) **Shortest processing time.** All jobs are sorted according to the descent order of their total process time, i.e. $T_j = \sum_{k=1}^S \bar{t}_{j,k}$, where $\bar{t}_{j,k}$ is the average processing time of the sub-lots of job j at stage k , as seen in [Eq. \(19\)](#).

$$\bar{t}_{j,k} = \frac{t_{j,k} \times L_j}{\bar{sp}_k} + \frac{\sum_{j'=1}^N setup_{k,j',j}}{N} + setup_{k,j,j} \times (L_j - 1) \quad (19)$$

where $\bar{sp}_k$ denotes the average speed coefficient of all machines at stage k .

- (2) **Johnson's rule.** Johnson's rule splits all jobs into two sets. The jobs meeting $p_{i,1} \leq p_{i,2}$ are assigned to the first set, and other jobs are assigned to the second set, where $p_{i,1}$ and $p_{i,2}$ are the average processing time of job j at the first stage and the second stage. The first set sorts the job in ascending order of the average processing time, and the second set sorts them in descending order of the average processing time. Then, these two sets are combined, and it can obtain a sequence. However, for S stages, they should be divided into two stages. Then, $S-1$ divided points can generate $S-1$ sequences. Stage 1 to stage k consist the first stage, and the rest stages consists the second stage. We evaluate these sequences by using the following job assignment heuristics, and the best sequence is selected. $p_{i,1}$ and $p_{i,2}$ can be computed by [Eq. \(20\)](#) and [Eq. \(21\)](#).

$$p_{j,1} = (1/k) \sum_{h=1}^k \frac{t_{j,h} \times L_j}{\bar{sp}_h} + \frac{\sum_{j'=1}^N setup_{h,j',j}}{N} + setup_{h,j,j} \times (L_j - 1) \quad (20)$$

$$p_{i,2} = (1/(S-k)) \sum_{h=k+1}^S \frac{t_{j,h} \times L_j}{\bar{sp}_h} + \frac{\sum_{j'=1}^N setup_{h,j',j}}{N} + setup_{h,j,j} \times (L_j - 1) \quad (21)$$

- (3) **Large medium rule.** As Johnson's rule, all jobs are sorted according to the descending order of $p_{i,2}$. Let the sorted sequence be $[\theta_1 \dots \theta_n]$. The first and the last job are extracted from $[\theta_1 \dots \theta_n]$ consecutively until all jobs have been checked. Then a new sequence is generated, i.e. $[\theta_1, \theta_n, \theta_2, \theta_{n-1}, \dots]$. As there are $S-1$ sequences, we also select the best sequence by using the following job assignment heuristics.
- (4) **Largest small rule.** This rule has one difference from Large medium rule, which selects the first and the middle jobs to establish the sequence $[\theta_1, \theta_{[n/2]}, \theta_2, \theta_{[n/2]+1}, \dots]$.
- (5) **Longest processing times till bottleneck stage.** The bottleneck stage is defined as the stage that has the largest the flow ratio $(1/m_k) \sum_{j=1}^N \bar{t}_{j,k}$. This rule sorts all jobs according to their average processing time from the first stage to the bottleneck stage in descending.
- (6) **KK.** This rule is based on the work of Kalczyński and Kamburowski (Kalczyński & Kamburowski, 2008). Firstly, the following indicators for each job are computed,

$$a_j = \sum_{k=1}^S [(S-1)(S-2)/2 + S - i] \times \bar{t}_{j,k} \quad (22)$$

$$b_j = \sum_{k=1}^S [(S-1)(S-2)/2 + k - i] \times \bar{t}_{j,k} \quad (23)$$

Secondly, all jobs are sorted according to the descending order of $\min(a_j, b_j)$.

In the NEH for distributed flow shop scheduling (called DNEH), each job is assigned to the factory that can process the job with the lowest completion time. To make a deep search, this paper designs two DNEH-based heuristics. As seen in Fig. 3, after a job κ is assigned into a position pos , the jobs before κ are assigned to the positions after pos . The best position that results in the lowest makespan of the current factory is selected. On the contrary, the jobs after κ are assigned to the position before pos in Fig. 3. These two heuristics reduce the search spacing compared to the above NEH-based algorithms. The procedures of $DNEH_{before}$ and $DNEH_{after}$ can be seen in Algorithm 1 and Algorithm 2 respectively.

Finally, we combine { Shortest processing time, Johnson's rule, Large medium rule, Largest small rule, Longest processing times till bottleneck stage, KK } and { DNEH, $DNEH_{before}$, $DNEH_{after}$ } and obtain eighteen constructive heuristics: H1 (Shortest processing time + DNEH), H2 (Shortest processing time + NEH_{before}), H3 (Shortest processing time + NEH_{after}), H4 (Johnson's rule + DNEH), H5 (Johnson's rule + NEH_{before}), H6 (Johnson's rule + NEH_{after}), H7 (Large medium rule + DNEH), H8 (Large medium rule + NEH_{before}), H9 (Large medium rule + NEH_{after}), H10 (Largest small rule + DNEH), H11 (Largest small rule + NEH_{before}), H12 (Largest small rule + NEH_{after}), H13 (Longest processing times till bottleneck stage + DNEH), H14 (Longest processing times till bottleneck stage + NEH_{before}), H15 (Longest processing times till bottleneck stage + NEH_{after}), H16 (Longest processing times till bottleneck stage + DNEH), H17 (Longest

processing times till bottleneck stage + NEH_{before}), H18 (Longest processing times till bottleneck stage + NEH_{after}).

Algorithm 1 $DNEH_{before}$

Input: job sequence σ generated by a heuristic
Output: a feasible solution π

- 1: **for** each job κ in σ **do**
- 2: try to insert κ into all possible positions of all factories.
- 3: insert κ into the factory of π that results in the lowest makespan after including κ .
- 4: record the factory and the position as f_r and pos
- 5: insert κ into position pos and record the makespan of factory f_r as δ
- 6: **if** $n^f > 2$ **then**
- 7: **for** $i = pos - 1$ **to** 1 **do**
- 8: remove job $\pi^f(i), pos_r = i$
- 9: **for** $j = pos + 1$ **to** n^f **do**
- 10: try to insert $\pi^f(i)$ to the positions j of factory f_r , and record this makespan as δ'
- 11: **if** $\delta' < \delta$ **then**
- 12: $pos_r = j$
- 13: $pos = pos - 1$, **break**
- 14: **end if**
- 15: **end for**
- 16: insert $\pi^f(i)$ to pos_r
- 17: **end for**
- 18: **end if**
- 19: **end for**

Algorithm 2 $DNEH_{after}$

Input: job sequence σ generated by a heuristic
Output: a feasible solution π

- 1: **for** each job κ in σ **do**
- 2: try to insert κ into all possible positions of all factories.
- 3: insert κ into the factory of π that results in the lowest makespan after including κ .
- 4: record the factory and the position as f_r and pos
- 5: insert κ into position pos and record the makespan of factory f_r as δ
- 6: **if** $n^f > 2$ **then**
- 7: **for** $i = pos + 1$ **to** n^f **do**
- 8: remove job $\pi^f(i), pos_r = i$
- 9: **for** $j = 1$ **to** pos **do**
- 10: try to insert $\pi^f(i)$ to the positions j of factory f_r , and record this makespan as δ'
- 11: **if** $\delta' < \delta$ **then**
- 12: $pos_r = j$
- 13: $pos = pos + 1$, **break**
- 14: **end if**
- 15: **end for**
- 16: insert $\pi^f(i)$ to pos_r
- 17: **end for**
- 18: **end if**
- 19: **end for**

5. Iterated local search algorithm for DHHFLSP

The iterated local search algorithm is a metaheuristic based on a local search method that uses a single solution in evolutionary process (Lour-enço, Martin, & Stützle, 2010). Generally, three sub-procedures, i.e. local search, perturbation, and acceptance criterion, form the core of any ILS algorithms. As seen in Algorithm 3, the process of ILS starts with a random initial solution or a promising heuristic within the predefined search space, then an alternative local search is applied to the initial solution. Next, a loop is performed until the stopping criterion is met. At each iterative, the perturbation is applied to the best feasible solution found so far, and then the local search is used to improve the solution. Despite its simplicity, the ILS algorithm has proven to be a very successful approach for solving shop scheduling problems (Jing, Pan, & Gao, 2021; Pan, Ruiz, & Alfaro-Fernández, 2017; Schulz, Neufeld, & Buscher, 2019; Shao et al., 2020). In the following sections, we describe the main components of ILS for the DHHFLSP including encoding and decoding, initialization, perturbation, local search, and acceptance criterion.

Algorithm 3 Iterated local search

Input: problem instance
Output: a feasible solution π^*

1. $\pi = \text{initialization}()$
2. $\pi = \text{local search}(\pi)$
3. $\pi^* = \pi$
4. **while** the terminate condition is not met **do**
5. $\pi' = \text{perturb}(\pi)$
6. $\pi' = \text{local search}(\pi')$
7. **if** $f(\pi') < f(\pi^*)$ **then**
8. $\pi^* = \pi'$
9. **end if**
10. $\pi = \text{acceptance}(\pi, \pi')$
11. **end while**
12. **return** π^*

5.1. Encoding and decoding

To simplify encoding, we employ the permutation-based encoding. The individual includes sequences. Each sequence represents the order

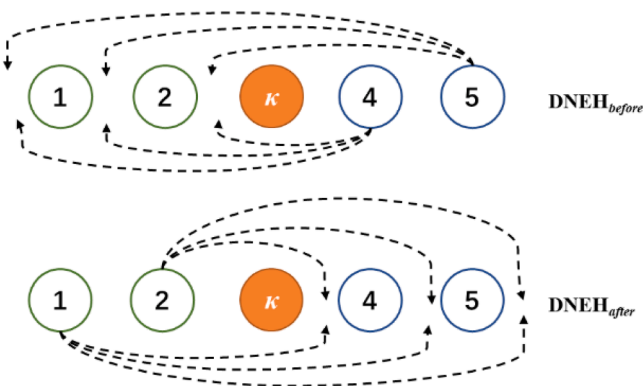


Fig. 3. Examples for $DNEH_{before}$ and $DNEH_{after}$.

of jobs in this sequence accessing the first stage of the current factories. After encoding, we would use some heuristics to transfer an individual to a feasible solution. This encoding method reduces solution space when searching and it may not cover absolute optimization solution. However, from the experimental results, it can also provide good solutions (Fernandez-Viagas, Perez-Gonzalez, & Framinan, 2019).

The decoding procedure should consider three issues: assignment sub-lots to factories, determination of order at each stage, and selection of machines at each stage. According to (Fernandez-Viagas et al., 2019), the combination of first input first out (FIFO) and first earliest machine (FAM) obtain the best performance for solving hybrid flow shop scheduling with the makespan criterion. In FIFO, the job completed first at a certain stage is given priority for production in the next stage. In FAM, the machines that are available first on a given stage are prioritized for production. However, the purpose of lot-streaming scheduling is to let the part of the job can early start to process at the next stage, so we consider if the first sub-lot is finished at a stage, then it would be prioritized to assign this sub-lot to the first earliest machine at the next stage. The sub-lot cannot be crossed, i.e. once the first sub-lot of a job arrives at a machine, other sub-lots of different jobs cannot be assigned to the machine until all of the sub-lots are processed (Zhang et al., 2017), as we consider the sequence dependent setup time between jobs and sub-lots. If the sub-lot from different jobs are crossed, the total setup time between jobs would increase (See Fig. 2).

The procedure of the decoding method for a job order $\pi^f = [\pi^f(1), \pi^f(2), \dots, \pi^f(n^f)]$ of in factory f can be described as follows.

Step 1. At the first stage $k = 1$, from the first job to the last one in π^f , denoted as $\pi^f(i)$,

Step 2. Select the machine that is first available and can process the job with the minimum completion time.

Step 3. Arrange each sub-lot of $\pi^f(i)$ to the machine, and compute the starting time and the completion time by considering the setup time and the speed coefficient.

Step 4. Update the available time of the machine with the completion time of last sub-lot of $\pi^f(i)$.

Step 5. For stage $k = 2$ to stage $k = S$, from the first job to the last one in π^f .

Step 6. Sort the jobs in π^f in ascending order the completion time of the first sub-lot of each job. If there are jobs that have the same completion time, the order is defined by π^f .

Step 7. For each job $\pi^f(i)$ in π^f , select the machine that is first available and can process the job with the minimum completion time.

Step 8. Arrange each sub-lot of $\pi^f(i)$ to the machine and compute the starting time and the completion time by considering the setup time and the speed coefficient.

Step 9. Update the available time of the machine with the completion time of the last sub-lot of $\pi^f(i)$.

5.2. Initialization

Usually, the initial solution of an ILS algorithm is generated randomly or by greedy construction heuristics (Lourengo et al., 2010). This paper selects a reasonable heuristic for the initialization of ILS. In Section 4, eighteen constructive heuristics are designed, and H8 and H9 are the two best methods among them. Therefore, we calibrate the initialization method by performing H8 and H9, and the method with the best results is selected as the initialization method.

5.3. Perturbation

The perturbation step is one of the key steps in ILS where a perturbation operator reconstructs the local optimal solution to find a diversified search space. Two issues should be focused on. The first is what techniques should be applied to perturb the solution, and the second is what should be the strength of the perturbation. This paper employs two

perturbation operators, i.e. P1 and P2, as seen in Algorithm 4 and Algorithm 5. In P1, several jobs are removed regardless of factories, then these jobs are re-inserted into the partial solution by a greedy insertion where the job will be tried to insert into all positions of factories. In P2, several jobs are removed from the critical factory since the job is not in the critical route, the makespan cannot be changed. The greedy insertion can ensure the perturbation solutions are not too poor.

Regarding the perturbation strength, i.e. the number of removed jobs, if it is too large, it may lead to a random start and decrease the convergence. If it is too small, the search may trap into the local optimum and the exploration of the search space will be limited within a small region (Xu, Lü, & Cheng, 2014). Therefore, the proposed ILS randomly selects a value as the perturbation strength from an interval $[d_{\min}, d_{\max}]$. To choose a proper strength, we design an experiment to calibrate it in Section 6.

Algorithm 4 P1	Algorithm 5 P2
Input: A feasible solution π	Input: A feasible solution π
Output: A reconstructed solution π	Output: A reconstructed solution π
1. Randomly remove d jobs from π regardless of factories, and add them into π_d . The remaining partial sequence is denoted as π_r .	1. Randomly remove d jobs from the critical factory of π , and update the critical factory after removing one job, then add them into π_d .
2. for each job λ in π_d do	2. The remaining partial sequence is denoted as π_r .
3. try to insert λ into all possible positions of all factories of π_r .	3. for each job λ in π_d do
4. record position pos that results in the minimum makespan.	4. try to insert λ into all possible positions of all factories of π_r .
5. insert λ into position pos	5. record position pos that results in the minimum makespan.
6. end for	6. insert λ into position pos
	7. end for

5.4. Local search

After the perturbation, a local search is carried out. A single type of local search cannot perform a deep search, so four local search methods are employed based on the critical factory which is the factory that has the maximum makespan. For the distributed hybrid flow shop scheduling, removing a job from the critical factory may not decrease the makespan, since the job may not be in the critical route. Therefore, we determine the effectiveness of the movement of jobs by computing the makespan of all factories.

- (1) LS1. Randomly remove a job from the critical factory and try to insert this job into all positions of all factories, then insert it into the position that results in the minimum makespan of all factories, finally update the critical factory. This procedure is repeated until all jobs in the critical factory have been checked. In heterogeneous factories, this operator is used to balance the task of each factory. The procedure of LS1 is shown in Algorithm 6.
- (2) LS2. Remove the job from the critical factory-one by one, and try to insert the job into all positions of the critical factory, and then insert it into the position that results in the minimum makespan of all factories, finally update the critical factory. This procedure is repeated until all jobs in the critical factory have been checked. This operator optimizes the sequence of the critical factory by updating the critical factory timely and changing the positions of jobs. The procedure of LS2 is shown in Algorithm 7.
- (3) LS3. Select each job from the critical factory-one by one, and try to swap the job with other jobs except the critical factory, and then swap it with the job that results in the minimum makespan of all factories, finally update the critical factory. This procedure is repeated until all jobs in the critical factory have been checked. This operator is also to balance each factory. The procedure of LS3 is shown in Algorithm 8.
- (4) LS4. Select each job from the critical factory-one by one, and try to swap the job with other jobs in the critical factory, and then

swap it with the job that results in the minimum makespan of all factories, finally update the critical factory. This procedure is repeated until all jobs in the critical factory have been checked. This operation is also used to reduce the makespan of the critical factory. The procedure of LS4 is shown in **Algorithm 9**.

As seen in **Algorithm 10**, LS1 ~ LS4 are performed inside a variable neighborhood descent method (Mladenović & Hansen, 1997). The first operator LS1 is performed on a solution. If the solution is not improved, the next operator LS2 is performed. If improvements are found, we go back to the first operator. This procedure is repeated iteratively until LS1 ~ LS4 have been checked.

Algorithm 6 LS1	Algorithm 7 LS2
Input: A feasible solution π Output: An improved solution π 1. $\sigma = \pi^c / f_c$ is the factory with maximum makespan*/ 2. while σ is not null then 3. random select a job λ from σ and remove it from σ 4. extract λ from π^c 5. try to insert λ to all positions of all factories and record position γ resulting in the minimum makespan $\min$ of all factories 6. if $\min < C_{\max}(\pi)$ then 7. insert λ to γ and update f_c 8. $\sigma = \pi^c$ 9. else 10. return λ into π^c 11. end if 12. end while	Input: A feasible solution π Output: An improved solution π 1. $\sigma = \pi^c / f_c$ is the factory with maximum makespan*/ 2. $count = 1$ 3. while $count \leq \pi^c $ do 4. remove $\pi^c(count)$ from π^c 5. try to insert $\pi^c(count)$ to all positions of f_c and record the position γ resulting in the minimum makespan $\min$ of all factories 6. if $\min < C_{\max}(\pi)$ then 7. $count = 1$, insert $\pi^c(count)$ to γ , update f_c 8. else 9. return $\pi^c(count)$ into π^c , $count = count + 1$ 10. end if 11. end while

Algorithm 8 LS3	Algorithm 9 LS4
Input: A feasible solution π Output: An improved solution π 1. $\sigma = \pi^c / f_c$ is the factory with maximum makespan*/ 2. $count = 1$ 3. while $count \leq \pi^c $ do 4. try to swap $\pi^c(count)$ with all jobs in all factories except f_c , record job γ resulting in the minimum makespan $\min$ of all factories 5. if $\min < C_{\max}(\pi)$ then 6. swap $\pi^c(count)$ and γ , update f_c , $count = 1$ 7. else 8. $count = count + 1$ 9. end if 10. end while	Input: A feasible solution π Output: An improved solution π 1. $\sigma = \pi^c / f_c$ is the factory with maximum makespan*/ 2. while $count \leq \pi^c $ do 3. try to swap $\pi^c(count)$ with all jobs of factory f_c , and record job γ resulting in the minimum makespan $\min$ of all factories 4. if $\min < C_{\max}(\pi)$ then 5. swap $\pi^c(count)$ and γ , update f_c , $count = 1$ 6. else 7. $count = count + 1$ 8. end if 9. end while

Algorithm 10 Local search
Input: A feasible solution π Output: An improved solution π 1. $\pi' = \pi$, $bestL = C_{\max}(\pi)$, $l = 1$, $l_{\max} = 4$ 2. while $l \leq l_{\max}$ do 3. if $l = 1$ then $\pi' = LS1(\pi')$ 4. else if $l = 2$ then $\pi' = LS2(\pi')$ 5. else if $l = 3$ then $\pi' = LS3(\pi')$ 6. else if $l = 4$ then $\pi' = LS4(\pi')$ 7. end if 8. if $bestL > C_{\max}(\pi')$ then 9. $bestL = C_{\max}(\pi')$, $l = l + 1$ 10. else 11. $l = l + 1$

(continued on next column)

(continued)

Algorithm 10 Local search

```

12. end if
13. end while
14. if  $C_{\max}(\pi') > C_{\max}(\pi)$  then
15.    $\pi = \pi'$ 
16. end if
17. return  $\pi$ 

```

5.5. Acceptance criterion

The acceptance criteria are used to determine whether the generated solutions are accepted for the next search. It can control the diversity of search to some degree. If more poor solutions are accepted for the next search, the searching may not be deep. This paper uses a simple acceptance criterion. Let π denote the solution that is used to perform perturbation and local search and π' denote the newly generated solution after performing perturbation and local search. If $f(\pi') < f(\pi)$, then $\pi = \pi'$. That is to say, the newly generated solution that is better than the current solution is accepted for the next iteration. In the testing results, we find that the heterogeneous factories have a great impact on the structure of solutions. We collect several good schedules in the results and find that these schedules are similar for a special testing instance, and the factory assignment of jobs and the order of jobs in each factory are relatively fixed, that is to say, the jobs always appear in a fixed position in the factory. It demonstrates that these collected schedules are around a local optimum. The search area does not jump larger and the search should focus the better solutions, since they are much closer to the optimal solutions. Therefore, only better solutions are accepted.

5.6. The framework of ILS for DHHFLSP

The whole procedure of ILS for the DHHFLSP is shown as **Algorithm 11**. First, an initialization solution is generated by a constructive heuristic. Then, two perturbation operators are sequentially performed. After one perturbation operator is finished, the perturbed solution is further improved by the local search procedure. If the improved solution is better than the previous solution, the above procedure would return to the perturbation part. This procedure is repeated until there is no improvement for the solution. If the new solution generated by the local search is better than the current solution, the current solution would be replaced by the new one.

Algorithm 11 Iterated local search for DHHFLSP

```

Input: problem instance, perturbation size  $d$ 
Output: a feasible solution  $\pi$ 
1.  $\pi =$  constructive heuristic /*initialization method*/
2.  $\pi' = \pi$ 
3. while the terminate condition is not met do
4.    $d = \text{random}[d_{\min}, d_{\max}]$ 
5.    $l = 1$ ,  $l_{\max} = 2$ 
6.   while  $l \leq l_{\max}$  do
7.     if  $(l = 1)$  then  $\pi' = P1(\pi', d)$  /*perturbation*/
8.     else if  $(l = 2)$  then  $\pi' = P2(\pi', d)$ 
9.      $\pi' = \text{Local search}(\pi')$ 
10.    if  $f(\pi') < f(\pi)$  then /*acceptance criterion*/
11.      $\pi = \pi'$ ,  $l = 1$ 
12.    else then
13.      $\pi' = \pi$ ,  $l = l + 1$ 
14.    end if
15.    end while
16.     $\pi = \pi'$ 
17. end while

```

6. Experiment evaluation

6.1. Experiment setup

To evaluate the performance of proposed algorithms and the effec-

tiveness of the mathematical model, three groups of testing instances are generated according to (Shao et al., 2021a):

- (1) The first group includes 600 large-sized instances that are used to measure the proposed constructive heuristics and ILS. Their configurations are $N \in \{20, 40, 60, 80, 100\}$, $S \in \{6, 8, 10, 12\}$, $F \in \{2, 3, 4, 5, 6, 7\}$, $t_{ij} \in [5, 20]$, $setup_{k,j,j'} \in [20, 30]$, $setup_{k,j,j} \in [1, 5]$, $setup_{f,k,j} \in [1, 10]$, $sp_{M_{k,j,i}} \in [0.8, 1.2]$, $L_j \in [2, 6]$, and $m_{f,k}$ refers to (Shao et al., 2021a). The combinations of N, S, F result 120 configurations, and each configuration is repeated five time.
- (2) The second group including 60 instances is used to calibrate the proposed ILS and compared algorithms. 60 configurations are randomly selected from the above configurations of the first group, and t_{ij} , $setup_{k,j,j'}$, $setup_{k,j,j}$, $sp_{M_{k,j,i}}$, L_j , $m_{f,k}$ are randomly generated in the same range.
- (3) Since the exact methods cannot solve large-sized instances, 36 small-sized instances with $N \in \{8, 10, 12\}$, $S \in \{2, 3, 4\}$, $F \in \{2, 3\}$ are generated. The ranges of other parameters are the same with the first group.

The relative deviation percentage (*RPD*) is used to evaluate the results, which can be computed by $RPD = (C_{\max}^i - C_{\max}^{best}) / C_{\max}^{best} \times 100\%$ where $C_{\max}^i$ is the makespan obtained by a testing algorithm in the i -th round for a specific instance and $C_{\max}^{best}$ denotes the best makespan of current instances. We collect the best values by counting all results obtained by all algorithms. Each algorithm for a specific instance is performed five times, and the average *RPD* values are used to compare. We re-implement all testing algorithms with Java (JDK 15) and run them under the same experiment environment, i.e. a PC (Lenovo Legion R7000P AMD 4000 series CPU, 16G RAM, and Windows 10 operation system). These algorithms share the same objective functions, encoding and decoding, uniform algorithm structure. All *meta*-heuristics terminate when the termination condition of the maximum elapsed CPU time of $100 \times N \times S$ ms is met. Moreover, the mean plot with 95 % intervals is used to demonstrate whether the difference is significant between algorithms. In the mean plot, if there are overlapping intervals, it demonstrates that the difference is not significant.

6.2. Evaluation of constructive heuristics

In Section 4, three types of job assignment rules ($DNEH$, $DNEH_{before}$, NEH_{after}) and six job priority ranking rules (*Shortest processing time*, *Johnson's rule*, *Large medium rule*, *Largest-small rule*, *Longest processing times till bottleneck stage*, *KK*) are described, then 18 constructive heuristics (H1 ~ H18) are provided. This sub-section tests the efficiency and effectiveness of these constructive heuristics. Table 1 lists the results obtained by eighteen heuristics. Fig. 4 shows the mean plot with 95 % LSD intervals of eighteen constructive heuristics.

From Table 1 and Fig. 4, the following conclusions can be observed. (1) H8 and H9 obtain the lower average *RPD* values for all groups compared to other heuristics. It demonstrates the effectiveness of the combination of *Large medium rule* and NEH_{before} and NEH_{after} . Therefore, we adopt H8 and H9 as the candidate initialization method and use a parameter calibration to determine which one is suitable for ILS. (2) The heuristics with $DNEH_{before}$ and NEH_{after} are better than $DNEH$, whose mean points are under $DNEH$. The reason may be $DNEH_{before}$ and NEH_{after} take much insertion time than $DNEH$. Although it may take more computing time, they obtain a better performance. (3) The sorting rules based on decomposition are better than other rules, since the average *RPD* values of H6 ~ H12 are lower than other heuristics. On the one hand, they increase the evaluation time and generate more candidate solutions. On the other hand, they find the best decomposition stage which benefits to finding the bottlenecks that affect the efficiency productivity. (4) *Longest processing times till bottleneck stage* rule with $DNEH$ is the worst heuristic compared to others. It sorts the jobs according to

Table 1
Average RPD values of eighteen constructive heuristics.

Instances	H1	H2	H3	H4	H5	H6	H7	H8	H9	H10	H11	H12	H13	H14	H15	H16	H17	H18
Factory																		
2	8.315	7.259	6.993	7.062	5.925	5.927	6.121	4.957	5.118	6.469	5.271	5.508	9.295	8.130	8.413	8.559	7.398	7.335
3	9.724	8.792	8.430	8.570	7.712	7.490	7.748	6.655	6.800	8.040	7.019	7.052	11.366	9.962	9.967	9.555	8.387	8.647
4	9.155	8.327	8.400	9.732	8.964	8.629	8.521	7.690	7.536	8.487	7.668	7.727	12.907	11.517	11.314	9.224	8.351	8.242
5	9.361	8.886	8.955	10.948	9.982	9.884	8.789	8.130	7.965	9.159	8.303	8.569	13.202	12.452	12.242	9.695	8.892	9.079
6	9.147	8.622	8.813	10.876	10.164	9.947	8.937	8.405	8.277	9.485	8.760	8.752	14.345	13.653	13.370	9.180	8.743	8.761
7	9.779	9.176	9.189	12.162	11.588	11.179	9.353	8.964	8.892	9.697	9.116	9.231	15.370	14.551	14.153	9.846	9.359	9.448
Job																		
20	11.729	11.583	11.076	13.059	12.526	12.290	11.053	10.612	10.338	11.340	10.830	10.962	18.724	18.144	17.456	11.779	11.476	11.209
40	10.058	9.385	9.122	10.943	10.117	9.972	8.930	8.221	8.367	9.468	8.628	8.677	13.876	12.758	12.535	10.048	9.339	9.607
60	8.851	8.037	8.270	9.780	8.820	8.499	8.065	7.169	7.034	8.349	7.376	7.561	11.894	10.636	10.680	9.106	8.089	8.374
80	8.247	7.250	7.350	8.158	7.340	7.062	7.014	6.074	6.132	7.240	6.135	6.315	10.423	9.270	9.118	8.178	7.431	7.269
100	7.350	6.297	6.500	7.518	6.476	6.390	6.162	5.258	5.285	6.384	5.477	5.518	8.820	7.744	8.092	7.605	6.273	6.469
Stage																		
6	10.007	9.271	9.281	10.866	9.956	9.693	9.334	8.510	8.427	9.602	8.818	8.778	13.413	12.588	12.265	10.129	9.382	9.530
8	9.479	8.946	8.867	10.447	9.664	9.523	8.573	7.865	7.704	8.977	8.004	8.205	13.250	12.304	12.000	9.507	8.711	8.719
10	9.001	8.203	8.109	9.569	8.689	8.529	8.061	7.038	7.231	8.239	7.493	7.577	11.424	11.436	11.424	9.185	8.351	8.326
12	8.500	7.622	7.597	8.684	7.914	7.625	7.012	6.455	6.362	7.408	6.441	6.667	11.575	10.515	10.616	8.551	7.642	7.767
Avg.	9.247	8.510	8.463	9.892	9.056	8.843	8.245	7.467	7.431	8.556	7.689	7.807	12.747	11.711	11.576	9.343	8.522	8.586

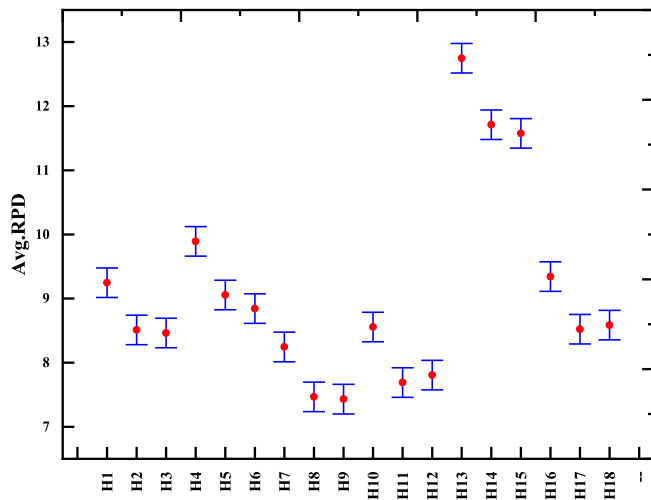


Fig. 4. Mean plots of eighteen constructive heuristics.

total flow time and it may be not suitable for lot-streaming scheduling.

6.3. Investigation of parameter setting

To maximize the efficiency of ILS, we calibrate the parameters and initialization methods of ILS. A full factorial experiment is carried out and the Analysis of Variance (ANOVA) method is used to investigate the results. We select two constructive heuristics, i.e. H8, and H9, as the candidate initialization methods of ILS (*initialType*). The proposed ILS has only one parameter, i.e. the interval of perturbation size (d). Seven candidate intervals are considered: [1, 1], [1, 3], [2, 4], [4, 6], [6, 8], [8, 12], [12, 16]. Calibrating methods on the same instances on which they are going to be tested later is bad practice and potentially unfair (Naderi & Ruiz, 2014). To avoid overfitting, 60 different instances are generated in Section 6.1, where the instance factors N , S and F are considered as noise factors and are not controlled in the experiment in order to avoid an instance specific calibration (Ruiz et al., 2019).

Table 2 lists the results of ANOVA for each parameter. Mean plot with 95 % LSD intervals for d and *initialType*.

d has the largest F -Ratio and its P -value is smaller than 0.05. It demonstrates the interval of d has a significant effect on the performance of ILS. From Fig. 5, when $d = 1$, the ILS obtains the worst results, while d in range [4, 6], it gets the best results. The value of d controls the diversity of searching. ILS with a smaller d results in trapping to local optimum, while a larger one leads to poor convergence. The influence of the initialization method is also significant. ILS with H8 obtains lower values compared to H9. Therefore, d is randomly generated from [4, 6] and the initial solution is generated by H8.

6.4. Evaluation of main components of ILS

The perturbation and local search parts are the critical parts of ILS, so this section tests the contribution and effectiveness of the main components of ILS. ILS includes two perturbation operators and four local search operators. We generate six variants of ILS by removing these operators. ILS1 and ILS2 only adopt one of $P1$ and $P2$ respectively. ILS3

Table 2
ANOVA for each parameter.

Source	Sum of Squares	Df	Mean Square	F-Ratio	P-Value
MAIN EFFECTS					
d	5470.15.0	6	9116.92	5199.58	0.0000
<i>initialType</i>	9.23733	1	9.23733	5.27	0.0217
INTERACTIONS					
$d * \text{initialType}$	7.40573	6	1.234	0.70	0.6464

~ ILS6 remove one of local search operators (LS1 ~ LS4) respectively. Table 3 shows the average RPD values of each variant of ILS. Fig. 6 displays the mean plot with 95 % LSD interval of all variants and ILS.

From Table 3 and Fig. 6, it can be obtained the following comments. (1) The total average RPD values of ILS are lower than other variants. It demonstrates each component makes contribution to the performance of ILS. (2) Regard as the perturbation operators, the average RPD values of $P2$ are better than that of $P1$. The reason may be that $P2$ always extracts jobs from the critical factories and this operation benefits to transferring the current searching to another space and jumping out of local optimality. (3) ILS3 obtains the lowest average RPD values and ILS5 obtains the highest value among ILS3 ~ ILS6. It demonstrates that the local searches in the critical factory play a larger role. The reason may be related to the properties of DHHFLSP. The heterogeneous factory may fix the allocation of certain jobs. If they are allocated to other factories, it will lead to larger makespan. Therefore, the orders in the factories are also even more important. In summary, the good performance of ILS can attribute to the main components.

6.5. Evaluation of MILP

This section verifies the effectiveness of MILP by using Gurobi (9.5). The small-sized instances mentioned in Section 6.1 are used since the exact methods cannot solve too large-sized problems. The maximum solving time of Gurobi is set to 1000 s. We collect the average RPD values and running time in Table 4. It can be seen from Table 4 that Gurobi obtains better results than ILS for some small-sized instances, but it takes much more time than ILS in solving small-sized problems. As the increasing of jobs, the performance of the exact algorithm decreases, while the results of ILS become better. The FIFO plus FAM rule and the encoding mechanism determine that the searching space of ILS cannot cover all solution space. The solutions do not include machine selection and the decoding method always selects the machines that results in the minimum completion time. This mechanism may not include the optimum solutions but it can generate some approximate solutions in a short running time.

6.6. Comparison of related algorithms

In this section, the proposed ILS is compared to some classic algorithms that are proposed for similar distributed flow shop scheduling problem, i.e. MIG (W. Shao, Shao, & Pi, 2020), IG2S (Ruiz et al., 2019), DABC (Y. Li et al., 2020), BCMA (Wang & Wang, 2020), and IG_W (Wang & Wang, 2019). MIG is the best algorithm for DHFSP with the makespan criterion. IG2S obtains the best results when solving the distributed permutation flow shop scheduling with the makespan criterion. DABC and BCMA are two population-based meta-heuristics proposed for DHFSP with the makespan criterion. IG_W is proposed for DHFSP with the total tardiness criterion. The reason for choosing these algorithms can be summarized as following two points.

- (1) There is no the state-of-art algorithm for DHHFLSP before our proposed method (ILS), since the DHHFLSP has not been considered before. Therefore, we select several high-efficient algorithms that are used to solve the distributed hybrid (or permutation) flow shop scheduling problem as the referenced algorithms (MIG, IG2S, DABC, BCMA, IG_W) to demonstrate the performance of ILS. These algorithms display good performance for solving approximate scheduling problems.
- (2) These algorithms are proposed for distributed hybrid (or permutation) flow shop scheduling problems, which adopt the same encoding mechanism and structure, i.e. multi-permutation list, so they can be easily extended to solve the DHHFLSP. Therefore, they can be regarded as the referenced algorithms.

To let these compared algorithms suitably solve the DHHFLSP, we must make some changes to these algorithms, such as evaluation

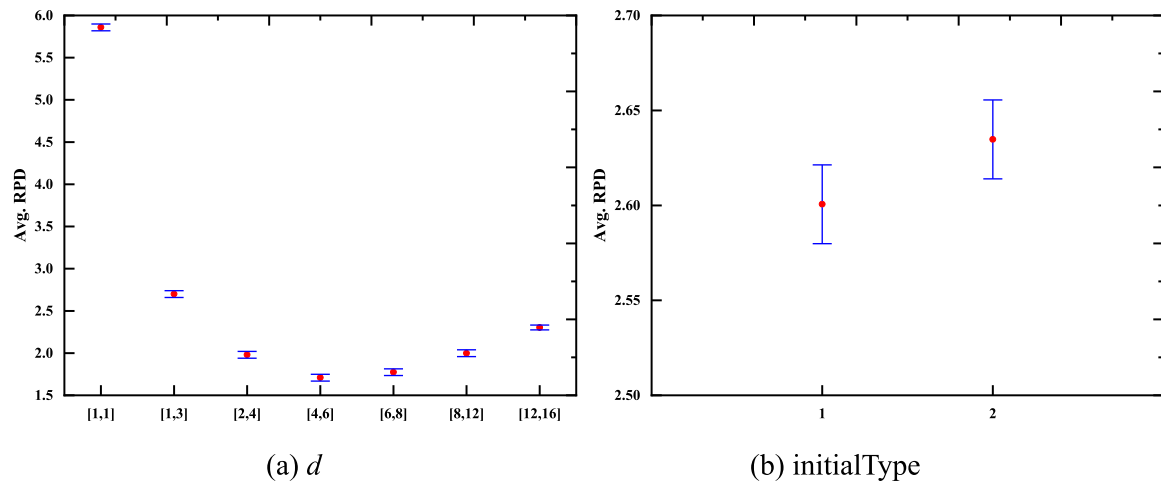


Fig. 5. Mean plot with 95% LSD intervals for and initialType.

Table 3

Average RPD values of each variants and ILS.

Instances	ILS1	ILS2	ILS3	ILS4	ILS5	ILS6	ILS
Factory							
2	1.608	1.534	1.546	1.470	1.796	1.634	1.447
3	1.828	1.624	1.617	1.600	2.046	1.809	1.615
4	1.815	1.579	1.560	1.600	2.028	1.803	1.478
5	1.916	1.631	1.580	1.709	2.147	1.890	1.533
6	1.674	1.576	1.567	1.479	2.097	1.704	1.323
7	1.843	1.840	1.769	1.538	2.369	1.783	1.481
Job							
20	1.711	2.019	1.968	1.634	2.412	1.699	1.522
40	1.818	1.615	1.614	1.550	2.042	1.778	1.484
60	1.794	1.507	1.521	1.656	2.083	1.826	1.587
80	1.840	1.582	1.525	1.555	2.038	1.876	1.456
100	1.740	1.431	1.404	1.435	1.828	1.675	1.348
Stage							
6	1.914	1.717	1.838	1.699	2.271	1.881	1.565
8	1.918	1.774	1.676	1.648	2.218	1.858	1.602
10	1.712	1.608	1.552	1.552	2.030	1.778	1.446
12	1.579	1.424	1.362	1.365	1.804	1.565	1.305
Avg.	1.781	1.631	1.607	1.566	2.081	1.771	1.479

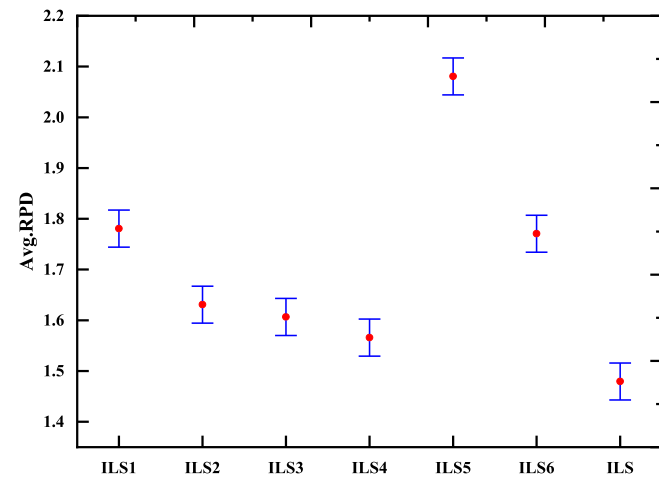


Fig. 6. Mean plot with 95% LSD interval of all variants and ILS.

Table 4

Average RPD values and running time of MILP and ILS.

Instances (N_S_F)	Average RPD		Running time	
	MILP	ILS	MILP	ILS
8_2_2	0.000	3.358	46.5	1
8_2_3	0.000	4.823	10.5	1
8_3_2	0.000	2.057	841	2
8_3_3	0.000	3.390	67	2
8_4_2	0.000	10.189	337.5	3
8_4_3	0.000	8.602	535	3
10_2_2	0.000	1.824	1000	2
10_2_3	0.000	0.826	1000	2
10_3_2	0.000	2.828	1000	3
10_3_3	3.374	0.069	1000	3
10_4_2	0.000	1.511	1000	4
10_4_3	0.393	0.000	1000	4
12_2_2	1.627	0.000	1000	2
12_2_3	2.692	0.398	1000	2
12_3_2	0.211	0.415	1000	3
12_3_3	0.950	1.116	1000	3
12_4_2	8.533	0.224	1000	4
12_4_3	7.986	0.000	1000	4

Table 5

The parameter settings of compared algorithms.

Algorithms	Parameter settings
BCMA	H8, Random greedy insertion initialization, $\alpha = 0.1, \eta = 0.15, LS = 100$
DABC	H8, Random greedy insertion initialization, $Psize = 10, \alpha = 30, \beta = 20, r = 0, T_0 = 0.4$
IG2S	IniType = H8, $d1 = 4, d2 = 6, T = 0.4$, DType = new, LSType = $LS3, \rho = 0.95$
IG_W	H8, $\alpha = 10, T_0 = 0.6$
MIG	H8, $d = 10, T_0 = 0.8$

functions, neighborhood structures, encoding and decoding. The parameters of all compared algorithms are also calibrated by the ANOVA methods and using the same testing instances. Due to the limited space of this paper, we only supplement the results of parameter settings of compared algorithms. The results of the parameters setting are shown in Table 5. The detail of ANOVA results can be seen in Supplementary materials.

Table 6 shows the results obtained by compared algorithms grouped by the number of factories, jobs, and stages. From this figure, it can be obtained the following conclusion. (1) The proposed ILS displays excellent performance in solving the DHHFLSP, as its average RPD

Table 6

Average RPD value of compared algorithms.

Instances	BCMA	DABC	IG2S	IG_W	MIG	ILS
Factory						
2	7.382	4.212	2.379	3.712	1.386	1.447
3	8.853	5.311	2.699	4.367	1.516	1.615
4	8.961	5.722	2.647	4.070	1.571	1.478
5	9.064	6.111	3.099	4.216	1.666	1.533
6	9.061	6.217	3.183	4.003	1.649	1.323
7	9.366	6.719	4.034	4.156	1.735	1.481
Job						
20	8.051	5.106	4.572	2.886	1.534	1.522
40	8.587	5.850	2.476	3.454	1.859	1.484
60	9.230	6.257	2.460	4.272	1.638	1.587
80	9.036	5.973	2.734	4.764	1.518	1.456
100	9.001	5.390	2.793	5.060	1.386	1.348
Stage						
6	9.639	6.151	3.270	4.205	1.661	1.565
8	9.113	5.825	3.100	4.230	1.634	1.602
10	8.460	5.636	2.906	4.083	1.642	1.446
12	7.912	5.250	2.752	3.832	1.411	1.305
Avg.	8.781	5.716	3.007	4.088	1.587	1.480

values are smaller than other algorithms. (2) The performance of MIG and ILS is very similar, as all their groups of RPD values are close to each other. However, ILS is slightly better than MIG. We think the efficient perturbation part and local search operators contribute more. (3) For the

single optimization problem, the deep search for the solution space plays an important role. It can be seen that the average RPD of IG2S, MIG, ILS are much lower than BCMA, DABC, IG_W which do not adopt greedy local search. This demonstrates the importance of deep local search.

Fig. 7 shows the mean plot with 95 % LSD interval for the interaction between algorithms and the numbers of job, stage, factory. From the figure, it can be obtained the following conclusions. (1) In Fig. 7 (d), the total mean plots of ILS locate under BCMA, DABC, IG2S, IG_W, MIG and the intervals of ILS do not overlap with each other. It demonstrates the best performance of ILS. (2) From the curve in Fig. 7 (a), (b), (c), ILS has the lowest average RPD values compared to BCMA, DABC, IG2S, IG_W in all groups. It demonstrates the good performance of ILS when solving different sized instances. The intervals of ILS slightly are close to MIG. It demonstrates the similar performance of ILS and MIG. (3) From the curve in Fig. 7 (c), the influence of the number of stages on the results is not obvious, as the curves are flat. In summary, the ILS can solve the DHHFLSP with its perturbation part and local search effectively and efficiently.

7. Conclusion and future work

This paper pays attention to a distributed heterogeneous hybrid flow shop lot-streaming scheduling problem, where the capability of factories is different and each job can be split into several sub-lots. Once the processing of a sub-lot is completed, it can be transferred to the next machine immediately. To solve this problem, we propose several constructive heuristics and an iterated local search algorithm. In these

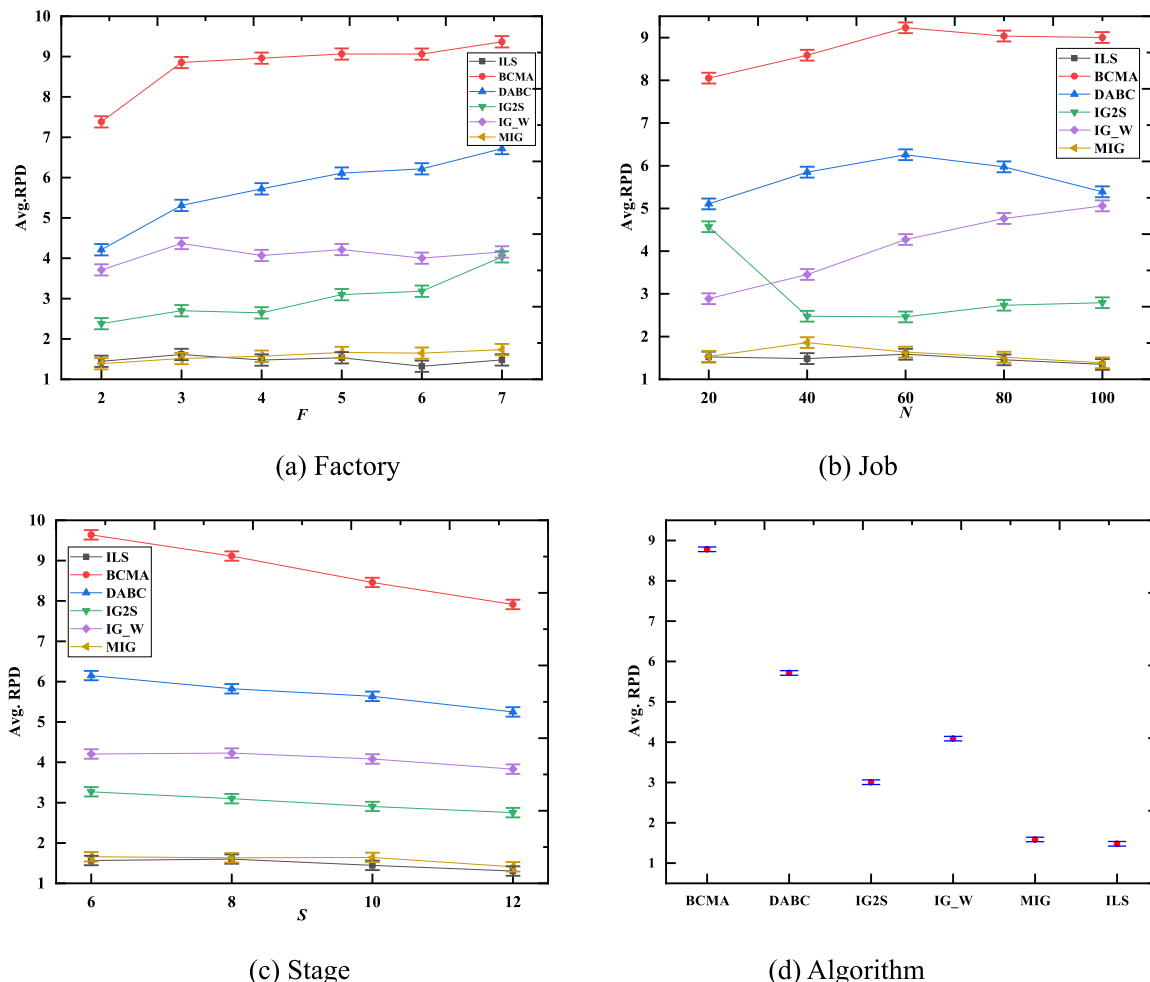


Fig. 7. Mean plot with 95% LSD interval for the interaction between algorithms and the numbers of jobs, stages, factories.

constructive heuristics, the jobs are sorted according to several heuristics rules or they are divided into several groups, and then these jobs are assigned into factories through several NEH-based algorithms. In the iterated local search algorithm, the best one in the proposed constructive heuristic, i.e. the NEH-based job assignment rule plus *Large medium rule*, is used to generate an initial solution. Two greedy insertion operators are adopted to generate the perturbation solutions. Then, four local search operators are presented to search solution space deeply. The comprehensive experiment tests the performance of ILS and the mathematical model of DHHFSP.

From the existing work and experiment results, we obtain the following interesting findings. (1) The critical bottleneck stage is important hybrid flow shop lot-streaming scheduling. We transverse all stages to find a critical stage, but the efficiency of this operation is not high where more candidate solutions are evaluated. (2) The deep searching would benefit to the single objective optimization, since the algorithms using greedy searching mechanism, such as ILS, IG2S, MIG obtain better results.

For the limitation of the proposed MILP and methods, it can be summarized as follows. The proposed MILP model employs position-indexed variables that represent all positions in machines. However, along with the increase of jobs, machines, and factories, the model may include amounts of binary variables. The studied problem only considered the different speeds and the different quantities of machines in distributed factories. These constraints are limited in real-life production. The utilization of the properties of schedules is limited. The proposed method only employs the jobs in the critical factory. However, waiting time, idle time, the quantity of sub-lots, or others may affect the schedule. The local search operators are limited. We only consider the movements of jobs in the order sequence and the factory assignment, however, the movements of jobs in the machines are also important.

For future work, the following directions can attract attention. Firstly, the distributed hybrid flow shop scheduling with lot-sizing is an interesting problem that combines the production plan and scheduling. Secondly, the study on the multi-objective distributed hybrid flow shop lot-streaming scheduling with energy-efficiency, cost, workers is another interesting. Thirdly, the adaptive adjustment techniques for ILS, such as parameters, perturbation operators, local search, are worth to explore. Fourthly, more properties of schedule are also worth to study.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgements

This research is supported by the National Natural Science Foundation of China (62103195, 62003203, 62262018), Jiangsu Natural Science Foundation (No. BK20210558), Youth Talent Support Program of Association for Science and Technology in Xi'an, China (No. 095920211321), China Postdoctoral Science Foundation funded project (No. 2021M701700), Fundamental Research Funds for the Central Universities (No. GK202201014) and Research Startup Fund of Shaanxi Normal University and Nanjing Normal University.

Appendix A. Supplementary data

Supplementary data to this article can be found online at <https://doi.org/10.1016/j.eswa.2022.119151>.

References

- Božek, A., & Werner, F. (2018). Flexible job shop scheduling with lot streaming and sublot size optimisation. *International Journal of Production Research*, 56, 6391–6411.
- Buscher, U., & Shen, L. (2009). An integrated tabu search algorithm for the lot streaming problem in job shops. *European Journal of Operational Research*, 199, 385–399.
- Cai, J., Zhou, R., & Lei, D. (2020). Dynamic shuffled frog-leaping algorithm for distributed hybrid flow shop scheduling with multiprocessor tasks. *Engineering Applications of Artificial Intelligence*, 90, Article 103540.
- Chang, J. H., & Chiu, H. N. (2005). A comprehensive review of lot streaming. *International Journal of Production Research*, 43, 1515–1536.
- Chen, T., Cheng, C., & Chou, Y. (2020). Multi-objective genetic algorithm for energy-efficient hybrid flow shop scheduling with lot streaming. *Annals of Operations Research*, 290, 813–836.
- Cheng, M., & Sarin, S. C. (2013). Two-stage, Multiple-lot, Lot Streaming Problem for a 1 + 2 Hybrid Flow Shop. *IFAC Proceedings Volumes*, 46, 448–453.
- Drexel, A., & Haase, K. (1995). Proportional lot sizing and scheduling. *International Journal of Production Economics*, 40, 73–87.
- Duan, J.-H., Meng, T., Chen, Q.-D., & Pan, Q.-K. (2018). An Effective Artificial Bee Colony for Distributed Lot-Streaming Flowshop Scheduling Problem. In *International Conference on Intelligent Computing* (pp. 795–806). Cham: Springer, Cham.
- Enda, J., Ling, W., & Jingjing, W. (2021). Decomposition-based multi-objective optimization for energy-aware distributed hybrid flow shop scheduling with multiprocessor tasks. *Tsinghua Science and Technology*, 26, 646–663.
- Fernandez-Viagas, V., Perez-Gonzalez, P., & Framinan, J. M. (2019). Efficiency of the solution representations for the hybrid flow shop scheduling problem with makespan objective. *Computers & Operations Research*, 109, 77–88.
- Fu, Y. P., Hou, Y. S., Wang, Z. F., Wu, X. W., Gao, K. Z., & Wang, L. (2021). Distributed scheduling problems in intelligent manufacturing systems. *Tsinghua Science and Technology*, 26, 625–645.
- Han, Y., Gong, D., Jin, Y., & Pan, Q. (2019). Evolutionary Multiobjective Blocking Lot-Streaming Flow Shop Scheduling With Machine Breakdowns. *IEEE Transactions on Cybernetics*, 49, 184–197.
- Hao, J., Li, J., Du, Y., Song, M., Duan, P., & Zhang, Y. (2019). Solving Distributed Hybrid Flowshop Scheduling Problems by a Hybrid Brain Storm Optimization Algorithm. *IEEE Access*, 7, 66879–66894.
- Jing, X.-L., Pan, Q.-K., & Gao, L. (2021). Local search-based metaheuristics for the robust distributed permutation flowshop problem. *Applied Soft Computing*, 105, Article 107247.
- Kalczynski, P. J., & Kamburowski, J. (2008). An improved NEH heuristic to minimize makespan in permutation flow shops. *Computers & Operations Research*, 35, 3001–3008.
- Lee, Y., & Lee, K. (2020). Lot-sizing and scheduling in flat-panel display manufacturing process. *Omega*, 93, Article 102036.
- Lei, D., & Wang, T. (2019). Solving distributed two-stage hybrid flowshop scheduling using a shuffled frog-leaping algorithm with memplex grouping. *Engineering Optimization*, 52, 1461–1474.
- Li, J. Q., Chen, X. L., Duan, P. Y., & Mou, J. H. (2022). KMOEA: A Knowledge-Based Multiobjective Algorithm for Distributed Hybrid Flow Shop in a Prefabricated System. *IEEE Transactions on Industrial Informatics*, 18, 5318–5329.
- Li, Y., Li, X., Gao, L., & Meng, L. (2020). An improved artificial bee colony algorithm for distributed heterogeneous hybrid flowshop scheduling problem with sequence-dependent setup times. *Computers & Industrial Engineering*, 147, Article 106638.
- Li, Y., Li, X., Gao, L., Zhang, B., Pan, Q., Tasgetiren, M. F., & Meng, L. (2021). A discrete artificial bee colony algorithm for distributed hybrid flowshop scheduling problem with sequence-dependent setup times. *International Journal of Production Research*, 59, 3880–3899.
- Li, J., Li, J., Zhang, L., Sang, H., Han, Y., & Chen, Q. (2021). Solving Type-2 Fuzzy Distributed Hybrid Flowshop Scheduling Using an Improved Brain Storm Optimization Algorithm. *International Journal of Fuzzy Systems*, 23, 1194–1212.
- Li, J.-Q., Tao, X.-R., Jia, B.-X., Han, Y.-Y., Liu, C., Duan, P., ... Sang, H.-Y. (2020). Efficient multi-objective algorithm for the lot-streaming hybrid flowshop with variable sub-lots. *Swarm and Evolutionary Computation*, 52, Article 100600.
- Lohmer, J., & Lasch, R. (2021). Production planning and scheduling in multi-factory production networks: A systematic literature review. *International Journal of Production Research*, 59, 2028–2054.
- Lourenço, H. R., Martin, O. C., & Stützle, T. (2010). Iterated Local Search: Framework and Applications. In *Handbook of Metaheuristics* (pp. 363–397). Springer.
- Lu, C., Gao, L., Yi, J., & Li, X. (2020). Energy-efficient scheduling of distributed flow shop with heterogeneous factories: A real-world case from automobile industry in China. *IEEE Transactions on Industrial Informatics*, 17, 6687–6696.
- Martinez, K. P., Adulyasak, Y., Jans, R., Morabito, R., & Toso, E. A. V. (2019). An exact optimization approach for an integrated process configuration, lot-sizing, and scheduling problem. *Computers & Operations Research*, 103, 310–323.
- Meng, T., & Pan, Q. (2021). A distributed heterogeneous permutation flowshop scheduling problem with lot-streaming and carryover sequence-dependent setup time. *Swarm and Evolutionary Computation*, 60, Article 100804.
- Mladenović, N., & Hansen, P. (1997). Variable neighborhood search. *Computers & Operations Research*, 24, 1097–1100.
- Naderi, B., & Ruiz, R. (2010). The distributed permutation flowshop scheduling problem. *Computers & Operations Research*, 37, 754–768.
- Naderi, B., & Ruiz, R. (2014). A scatter search algorithm for the distributed permutation flowshop scheduling problem. *European Journal of Operational Research*, 239, 323–334.

- Naderi, B., & Yazdani, M. (2014). A model and imperialist competitive algorithm for hybrid flow shops with sublots and setup times. *Journal of Manufacturing Systems*, 33, 647–653.
- Nejati, M., Mahdavi, I., Hassanzadeh, R., & Mahdavi-Amiri, N. (2016). Lot streaming in a two-stage assembly hybrid flow shop scheduling problem with a work shift constraint. *Journal of Industrial and Production Engineering*, 33, 459–471.
- Pan, Q.-K., Fatih Tasgetiren, M., Suganthan, P. N., & Chua, T. J. (2011). A discrete artificial bee colony algorithm for the lot-streaming flow shop scheduling problem. *Information Sciences*, 181, 2455–2468.
- Pan, Y., Gao, K., Li, Z., & Wu, N. (2022a). Improved meta-heuristics for solving distributed lot-streaming permutation flow shop scheduling problems. *IEEE Transactions on Automation Science and Engineering*. Early Access.
- Pan, Y., Gao, K., Li, Z., & Wu, N. (2022b). Solving biobjective distributed flow-shop scheduling problems with lot-streaming using an improved Jaya algorithm. *IEEE Transactions on Cybernetics*. Early Access.
- Pan, Q.-K., & Ruiz, R. (2012). An estimation of distribution algorithm for lot-streaming flow shop problems with setup times. *Omega*, 40, 166–180.
- Pan, Q.-K., Ruiz, R., & Alfaro-Fernández, P. (2017). Iterated search methods for earliness and tardiness minimization in hybrid flowshops with due windows. *Computers & Operations Research*, 80, 50–60.
- Qin, W., Zhuang, Z., Liu, Y., & Tang, O. (2019). A two-stage ant colony algorithm for hybrid flow shop scheduling with lot sizing and calendar constraints in printed circuit board assembly. *Computers & Industrial Engineering*, 138, Article 106115.
- Ruiz, R., Pan, Q., & Naderi, B. (2019). Iterated Greedy methods for the distributed permutation flowshop scheduling problem. *Omega*, 83, 213–222.
- Schulz, S., Neufeld, J. S., & Buscher, U. (2019). A multi-objective iterated local search algorithm for comprehensive energy-aware hybrid flow shop scheduling. *Journal of Cleaner Production*, 224, 421–434.
- Shao, W., Pi, D., & Shao, Z. (2017). Optimization of makespan for the distributed no-wait flow shop scheduling problem with iterated greedy algorithms. *Knowledge-Based Systems*, 137, 163–181.
- Shao, Z., Shao, W., & Pi, D. (2020). Effective constructive heuristic and metaheuristic for the distributed assembly blocking flow-shop scheduling problem. *Applied Intelligence*, 50, 4647–4669.
- Shao, W., Shao, Z., & Pi, D. (2020). Modeling and multi-neighborhood iterated greedy algorithm for distributed hybrid flow shop scheduling problem. *Knowledge-Based Systems*, 194, Article 105527.
- Shao, W., Shao, Z., & Pi, D. (2021a). An ant colony optimization behavior-based MOEA/D for distributed heterogeneous hybrid flow shop scheduling problem under nonidentical time-of-use electricity tariffs. *IEEE Transactions on Automation Science and Engineering*. Early Access.
- Shao, W., Shao, Z., & Pi, D. (2021b). Effective constructive heuristics for distributed no-wait flexible flow shop scheduling problem. *Computers & Operations Research*, 136, Article 105482.
- Shao, W., Shao, Z., & Pi, D. (2021c). Multi-objective evolutionary algorithm based on multiple neighborhoods local search for multi-objective distributed hybrid flow shop scheduling problem. *Expert Systems with Applications*, 183, Article 115453.
- Wang, J., & Wang, L. (2019). An iterated greedy algorithm for distributed hybrid flowshop scheduling problem with total tardiness minimization. In *2019 IEEE 15th International Conference on Automation Science and Engineering (CASE)* (pp. 350–355). Vancouver, BC, Canada: IEEE.
- Wang, J., & Wang, L. (2020). A bi-population cooperative memetic algorithm for distributed hybrid flow-shop scheduling. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 5, 947–961.
- Wang, J., & Wang, L. (2021). A cooperative memetic algorithm with learning-based agent for energy-aware distributed hybrid flow-shop scheduling. *IEEE Transactions on Evolutionary Computation*, 26, 461–475.
- Xu, H., Lü, Z., & Cheng, T. C. E. (2014). Iterated Local Search for single-machine scheduling with sequence-dependent setup times to minimize total weighted tardiness. *Journal of Scheduling*, 17, 271–287.
- Ying, K.-C., & Lin, S.-W. (2018). Minimizing makespan for the distributed hybrid flowshop scheduling problem with multiprocessor tasks. *Expert Systems with Applications*, 92, 132–141.
- Zhang, B., Pan, Q.-K., Gao, L., Zhang, X.-L., Sang, H.-Y., & Li, J.-Q. (2017). An effective modified migrating birds optimization for hybrid flowshop scheduling problem with lot streaming. *Applied Soft Computing*, 52, 14–27.
- Zhang, W., Yin, C., Liu, J., & Linn, R. J. (2005). Multi-job lot streaming to minimize the mean completion time in m-1 hybrid flowshops. *International Journal of Production Economics*, 96, 189–200.
- Zheng, J., Wang, L., & Wang, J. (2020). A cooperative coevolution algorithm for multi-objective fuzzy distributed hybrid flow shop. *Knowledge-Based Systems*, 105536.
- Zohali, H., Naderi, B., Mohammadi, M., & Roshanaei, V. (2019). Reformulation, linearization, and a hybrid iterated local search algorithm for economic lot-sizing and sequencing in hybrid flow shop problems. *Computers & Operations Research*, 104, 127–138.